第 4 章 数据依赖与关系模式规范化

西安交通大学计算机科学与技术学院　　任雪斌

xuebinren@mail.xjtu.edu.cn

第 4 章 数据依赖与关系模式规范化



4.1 问题的提出

4.2 数据依赖

4.3 关系模式分解

4.4 关系模式规范化

4.1 问题的提出

- 关系模式设计中的问题
  - 前面我们已经讨论了关系数据库的基本概念、关系模型的三个部分以及关系数据库的标准语言 SQL
  - 但是，还有一个基本问题尚未涉及，即针对一个具体的应用问题，应该如何构造一个适合于它的关系数据库模式？应该构造几个关系模式？每个关系由哪些属性组成？
  - 本章的任务是研究设计一个“好”的（即没有“毛病”的）关系模式的方法

4.1 问题的提出

➢实例分析

- 前面章节中多次引用过的三个基本表：

  Student(S# ，Class ，Sname ，...)、

  Course(C# ，Cname ，Teacher)、

  SC(S# ，C# ，Grade)

- 对于这个应用而言，为什么仅设计了这三个表，如果将这三个表合并成一个或两个表，或者将其拆分为更多的表，这样是否更合适？

4.1 问题的提出

➢实例分析

•考虑下面的关系模式 SCT:

SCT(S#,C#,Grade,Teacher,Dept)

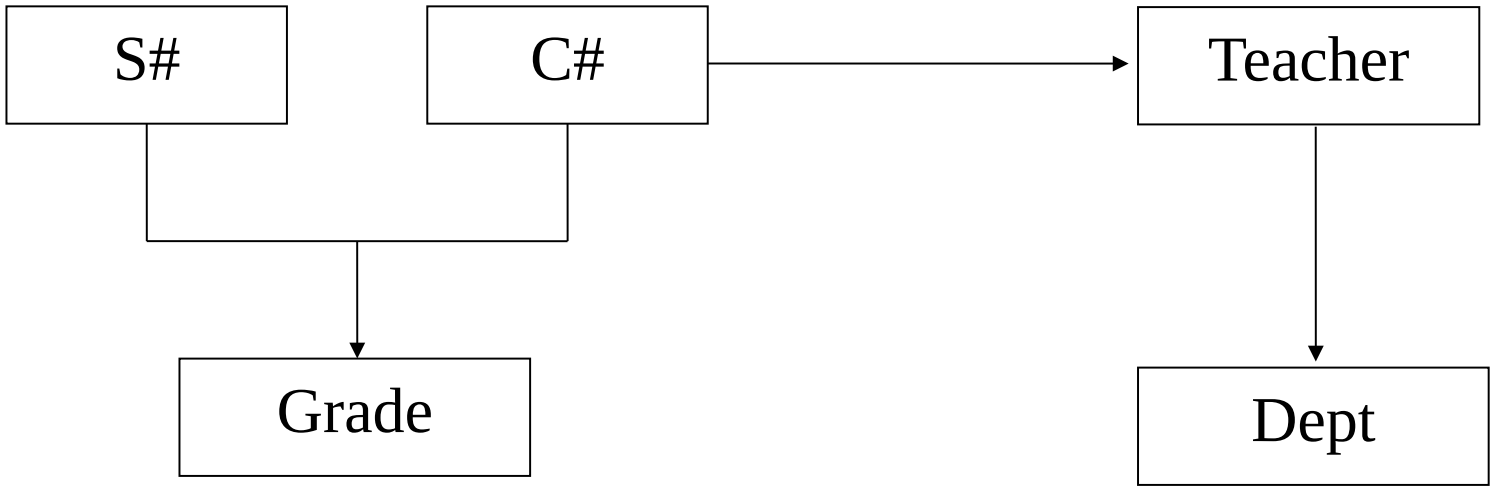


图 4.1 关系模式 SCT 属性间依赖关系

4.1 问题的提出

➢实例分析

S#	C#	Grade	Teacher	Dept
020550 19	CS - 01	89	刘朝	计算机
	CS - 03	86	张莹	计算机
020550 19	CS - 01	68	刘朝	计算机
	CS - 02	92	林琳	计算机
020550 37	CS - 03	73	张莹	计算机
	…	…	…	…
020550 66				
020550 66				
…				

图 4.2 关系模式 SCT 实例

4.1 问题的提出

- 实例分析
  - 关系模式 SCT 存在的问题：
    - 数据冗余 (Redundancy)
    - 更新异常 (Update Anomalies)
    - 插入异常 (Insertion Anomalies)
    - 删除异常 (Deletion Anomalies)

4.1 问题的提出

- 实例分析
  - 结论：
    - SCT 不是一个”好”的关系模式
    - “好”的模式：数据冗余应尽可能少，不会发生插入异常、删除异常、更新异常
  - 原因：
    - 由存在于模式中的某些”数据依赖”引起的
  - 解决方法：
    - 通过分解关系模式来消除其中不合适的数据依赖

4.2 数据依赖

- 数据依赖 **(Data Dependency)** 是通过一个关系中各属性值相等与否体现出来的一种数据间的相互制约关系，是现实世界中事物属性间相互联系的抽象，是数据内在的性质，是语义的体现
- 人们已经提出了许多种类型的数据依赖，其中最重要的是函数依赖 **(Functional Dependency** 简记为 **FD)** 和多值依赖 **(Multivalued Dependency** 简记为 **MVD)**

4.2 数据依赖

常用符号及其含义

- R表示一个抽象的关系模式， R上全部函数依赖的集合记为F；
- R的属性全集一般用U表示， $\mathrm{U} = \{A_1, A_2, \ldots, A_n\}$ ， 其中的每一个 $A_i(1 \leq i \leq n)$ 均表示R的一个属性， X、Y、Z等表示U的一个真子集，也就是若干属性的集合；

4.2 数据依赖

常用符号及其含义

- 关系模式可以表示为：$R(U, D, DOM, I, F)$
- *例子：有一个关系模式为：*$STUDENT(U, D, DOM, I, F)$

- 属性集合$U = \{sno, name, age\}$　　//理解为表的字段集合
- 域集合$D = \{char, int\}$　　// 理解为表中字段的类型集合
- 属性到域的映射集合$DOM = \{dom(sno) = dom(name) = char,\ dom(age) = int\}$　　// 理解为每个字段具体的取值
- 完整性约束集合$I = \{sno! = Null,\ IsUnique(sno) = True,\ 18 <= age <= 22\}$

  //理解为表字段取值的限制

4.2 数据依赖

➢关系的内涵和外延

- 关系的内涵是对数据的定义以及数据完整性约束的定义，一般是与时间独立的
- 对数据的定义包括对关系、属性、域的定义和说明。对数据完整性约束的定义涉及面较广，主要包括以下几个方面：
  - 静态约束，涉及到数据之间联系（称为“数据依赖， data dependences ）、主键和值域的设计
  - 动态约束，定义各种操作（插入、删除、修改）对关系值的影响

4.2 数据依赖

函数依赖的定义

- 定义4.1：设R为关系模式，r是R上的任意一个关系实例，$X, Y \subseteq U$是R的两个属性子集，若对于r上的任意两个元组$t_1, t_2 \in r$都有：如果$t_1[X] = t_2[X]$，则必有$t_1[Y] = t_2[Y]$，则称在R上X函数决定Y或者Y函数依赖于X，记为$X \rightarrow Y$，X称为决定子(Determinant)

4.2 数据依赖

➢函数依赖的定义

- 函数依赖不是指关系模式 R 的某个或某些关系实例满足的约束条件，而是指 R 的所有关系实例均要满足的约束条件。
- 函数依赖是语义范畴的概念，只能根据数据的语义来确定函数依赖是否成立。
- *例：对于学生关系*

  学生（学号，姓名，性别，出生日期，班级，宿舍，专业）

  班级→性别 是否成立？

  如果规定“男女生分班”，则成立

  否则，不成立。

4.2 数据依赖

函数依赖的定义

- 定义4.2：设R为关系模式，X、Y是R的不同属性集，如果$X \to Y$成立，且不存在$X' \subset X$使得$X' \to Y$也成立，则称Y完全函数依赖于X，记为$X \xrightarrow{f} Y$，否则称Y部分依赖于X，记为$X \xrightarrow{P} Y$
- 完全函数依赖中的决定子不包含冗余属性，即只要将决定子中的任何一个属性去掉，这个函数依赖就不再成立了
- 定义4.3：如果$Y \subseteq X$,则称$X \to Y$为平凡的函数依赖

4.2 数据依赖

➢非平凡函数依赖

- 定义4.2：在关系模式$R(U)$中，对于U的子集X和Y：

  如果$X \to Y$，且$Y \subseteq X$，则称$X \to Y$是平凡的函数依赖

  如果$X \to Y$，且$Y \nsubseteq X$，则称$X \to Y$是非平凡的函数依赖

  对于任一关系模式，平凡函数依赖都是必然成立的，它不反映新的语义，因此若不特别声明，我们总是讨论非平凡函数依赖。

- 例：在关系$SC(Sno, Cno, Grade)$中，

  非平凡函数依赖：$(Sno, Cno) \to Grade$

  平凡函数依赖：$(Sno, Cno) \to Sno$

4.2 数据依赖

完全/部分函数依赖

- 定义4.3：在关系模式$R(U)$中，如果$X \to Y$，并且对于X的任何一个真子集X'，都有$X' \not\to Y$，则称Y完全函数依赖于X，记作X $\xrightarrow{F}$ Y。若$X \to Y$，但Y不完全函数依赖于X，则称Y部分函数依赖于X，记作X $\xrightarrow{P}$ Y。完全函数依赖中的左部不包含冗余属性，即只要将决定子中的任何一个属性去掉，这个函数依赖就不再成立了
- *例：在关系$SC(Sno, Cno, Grade)$中，由于：$Sno \to Grade$，$Cno \to Grade$，因此：$(Sno, Cno) \xrightarrow{F} Grade$*

4.2 数据依赖

➢函数依赖的定义

- 定义4.4：设X、Y、Z是R上的不同属性集合，如果有$X \to Y$，$Y \to Z$成立且$Y \not\to X$，则称Z传递函数依赖于X
- 条件$Y \not\to X$非常重要，如果没有这个条件，那么X与Y就是一一对应关系，从而$X \to Z$就是直接函数依赖
- *例: 在关系$Std(Sno, Sdept, Mname)$中，有:*

  $Sno \to Sdept$，$Sdept \to Mname$

  因此，$Mname$传递函数依赖于Sno

4.2 数据依赖

函数依赖的推理规则

- 设U是关系模式R的属性集，F是R上的函数依赖集，则有：
  - A1（自反率）：如果$Y \subseteq X \subseteq U$，则$X \rightarrow Y$成立
  - A2（增广率）：如果$X \rightarrow Y$成立，且$Z \subseteq U$，则$X \rightarrow YZ$成立
  - A3（传递率）：如果$X \rightarrow Y, Y \rightarrow Z$成立，则$X \rightarrow Z$成立
- 人们又把自反率（A1），增广率（A2）和传递率（A3）称为Armstrong公理

4.2 数据依赖

引理4-1：$Armstrong$公理是正确的

证： A1中的$X \to Y$其实就是平凡的函数依赖，我们仅对A2、A3进行证明：

A2：设t_1，t_2是关系模式R的任一关系实例r上的两个元组，如果$t_1[XZ] = t_2[XZ]$成立，则一定有$t_1[X] = t_2[X]$及$t_1[Z] = t_2[Z]$也成立。根据已知$X \to Y$及$t_1[X] = t_2[X]$可得$t_1[Y] = t_2[Y]$，于是可知$t_1[Y] = t_2[Y]$与$t_1[Z] = t_2[Z]$同时成立，即$t_1[YZ] = t_2[YZ]$，由定义4-1易知$XZ \to YZ$成立。

A3：设t_1，t_2是关系模式R的任一关系实例r上的两个元组，如果$t_1[X] = t_2[X]$成立，则根据$X \to Y$可得$t_1[Y] = t_2[Y]$，又因为$Y \to Z$也成立，所以有$t_1[Z] = t_2[Z]$，结合$t_1[X] = t_2[X]$，$t_1[Z] = t_2[Z]$及定义4-1可知$X \to Z$成立。

4.2 数据依赖

函数依赖的推理规则

- 由A1～A3易推出下面的三条推理规则也是正确的：
  - 合并规则：若$X \to Y$，$X \to Z$成立，则$X \to YZ$成立
  - 伪传递规则：若$X \to Y$，WY $\to Z$成立，则W$X \to Z$成立
  - 分解规则：若$X \to Y$，且$Z \subseteq Y$，则$X \to Z$
- A2可以由合并规则代替

4.2 数据依赖

函数依赖的推理规则

- 引理4.1：FD推理规则A1、A2和A3是正确的
- 也就是，如果$X \to Y$是从F用推理规则导出，那么$X \to Y$在F^+（*）
- 由A1～A3易推出下面的三条推理规则也是正确的：
  - 合并规则：若$X \to Y$，$X \to Z$成立，则$X \to YZ$成立
  - 伪传递规则：若$X \to Y$，$WY \to Z$成立，则$WX \to Z$成立
  - 分解规则：若$X \to Y$，且$Z \subseteq Y$，则$X \to Z$
- A2可以由合并规则代替

4.2 数据依赖

函数依赖的逻辑导出

- 定义4.7：给定关系模式$R<U,F>$，如果能由F根据Armstrong公理导出$X \rightarrow Y$，则称$X \rightarrow Y$是F的逻辑导出，记为$F \Longrightarrow X \rightarrow Y$

4.2 数据依赖

函数依赖的逻辑导出

- 定义4.5：设F是在关系模式R上成立的函数依赖的集合，$X \to Y$是一个函数依赖，如果对于R的每个满足F的关系r也满足$X \to Y$，那么称F逻辑蕴含$X \to Y$，记为$F \vDash X \to Y$
- 例如：$\{X \to Y, Y \to Z\} \vDash X \to Z$

4.2 数据依赖

函数依赖集合的闭包

- 定义4.6：由函数依赖集合F所逻辑蕴涵的全部函数依赖所构成的集合称之为F的闭包（closure），记作F^+，即$F^+ = \{X \to Y | F \vDash X \to Y\}$
- 据定义4.6易知，函数依赖集的闭包F^+具有以下特点：
  - $F \subseteq F^+$，这是因为根据闭包的定义F中的每个函数依赖必定也在F^+中；
  - $(F^+)^+ = F^+$，该性质说明闭包运算是幂等（idempotence）的，即F经过任意多次闭包运算后其结果依然等于F^+；
  - 如果$F = F^+$，则称F是完备的。

4.2 数据依赖

函数依赖集合的闭包

- 例4.1. 计算$R(A,B,C)$上的函数依赖集合$F=\{A\to B, B\to C\}$的闭包：

$$F^{+}=\left\{\begin{array}{c} A\to B, B\to C, A\to C \\ A\to A, B\to B, C\to C \\ AB\to A, AB\to B, AB\to AB\ldots \end{array}\right\}$$

- F^{+}的计算是一个NP完全问题

4.2 数据依赖

属性集合的闭包

- 定义4.8：属性集X关于$R(U,F)$上的函数依赖集合F的闭包${X_F}^+$定义为${X_F}^+ = \{A|A \in U, F \Longrightarrow X \to A\}$。
- 根据A1（自反率）可知对于任一属性集X一定有$X \subseteq {X_F}^+$，此外，计算${X_F}^+$的工作量应该远远小于计算F^+的工作量，因为${X_F}^+$中的属性一定是可数的，最多就是属性全集U而已。

4.2 数据依赖

属性集合的闭包

- 引理4.2：$X \to Y$可由Armstrong公理推出的充分必要条件是$Y \subseteq {X_F}^+$。

证：

充分性：已知$Y \subseteq {X_F}^+$成立，要证$X \to Y$可由Armstrong公理推出。设$Y = B_1B_2...B_n$，则有$B_1B_2...B_n \subseteq {X_F}^+$，于是有$B_i \subseteq {X_F}^+(1 \le i \le n)$，按照定义4.6可知$X \to B_i$可由Armstrong公理推出，再根据合并规则及$i$的任意性可得$X \to B_1B_2...B_n$可由Armstrong公理推出，即$X \to Y$可由Armstrong公理推出；

必要性：已知$X \to Y$可由Armstrong公理推出，要证$Y \subseteq {X_F}^+$。类似于充分性的证明，利用分解规则可证。

4.2 数据依赖

属性集的闭包

- 算法4.1：求属性集$X(X \subseteq U)$关于U 上的函数依赖集F的闭包${X_F}^+$
  - 输入： X, F
  - 输出${X_F}^+$
  - 步骤：
    1）令$X(0) = X, i = 0$
    2）求$B = \{A|(\text{存在}V \to W)(V \to W \in F \wedge V \subseteq X(i) \wedge A \in W)\}$;
    3）更新$X(i+1) = X(i) \cup B$
    4）判断$X(i+1) = X(i)$吗?
    5）若相等或$X(i) = U$，则$X(i)$就是${X_F}^+$，算法终止。
    6）否则$i = i + 1$，返回第2）步。

4.2 数据依赖

属性集的闭包

- 例4.2:已知关系模式$R<U,F>$, 其中:$U=\{A,B,C,D,E\}$； $F=\{AB\to C$， $B\to D$， $C\to E$， $EC\to B$， $AC\to B\}$，求 $(AB)+F$ 。
- 解： 由算法， $X(0)=AB$； 计算$X(1)$； 逐一的扫描F集合中各个函数依赖,找左部为A,B,或AB的函数依赖。得到两个： $AB\to C$， $B\to D$。于是$X(1)=AB\cup CD=ABCD$。

  因为 $X(0)\neq X(1)$， 所以再找出左部为$ABCD$子集的那些函数依赖， 又得到$C\to E$， $AC\to B$， 于是$X(2)=X(1)\cup BE=ABCDE$。

  因为$X(2)$ 已等于全部属性集合,所以$(AB)+F=ABCDE$

  $(BC)+F=?$ $(AC)+F=?$
- 注意： 对于算法1.1， 令$ai=|X(i)|$,$\{ai\}$形成一个步长大于1的严格递增的序列,序列的上界是$|U|$, 因此该算法最多$|U|-|X|$次循环就会终止。

4.2 数据依赖

属性集的闭包

- 例4.3: 属性集U为$ABCD$，FD集为 $\{A \to B，\ B \to C，\ D \to B\}$
- 则用上述算法，可求出:
- $A^{+} = ABC$，

  $AD^{+} = ABCD$

  $BD^{+} = BCD$

  等等

4.2 数据依赖

Armstrong公理的正确性和完备性

- Armstrong公理的正确性是指“使用推理规则从FD集F推出的FD必定在F^+中”
  即，如果$F \Longrightarrow X \rightarrow Y$，则$F \vDash X \rightarrow Y$
- Armstrong公理的完备性是指“F^+中的FD都能从F集使用推理规则集导出”
  即，如果$F \vDash X \rightarrow Y$，则 $F \Longrightarrow X \rightarrow Y$
- Armstrong公理的正确性和完备性保证了FD推导过程的有效性和可靠性

4.2 数据依赖

函数依赖集的等价覆盖

- 定义4.8：设F,G是两个函数依赖集合，如果$F^+ = G^+$，则称F等价于G，或者F,与G互相覆盖
- 引理4.3： F与G等价的充分必要条件是$F \subseteq G^+$并且$G \subseteq F^+$

证：必要性显然，只需证充分性：

1)若$F \subseteq G^+$,则X^+_F $X^+_{G^+}$

2)任取$X \to Y \in F^+$ 则有 $Y \subseteq X^+_F$ $X^+_{G^+}$ 。 所以$X \to Y \in (G^+)^+ = G^+$ ，即$F^+ \subseteq G^+$

3)同理可证$G^+ \subseteq F^+$ ，所以$F^+ = G^+$

4.2 数据依赖

函数依赖集的等价覆盖

- 要判定$F \subseteq G^+$,只须逐一对F中的函数依赖$X \to Y$，考查Y是否属于$X^+{}_G$
- 引理4. 3给出了判断两个函数依赖集等价的可行算法

4.2 数据依赖

最小函数依赖集

- 定义4.9：若F满足下列条件：

(1) F中所有函数依赖的右部均为单属性；

(2) F中不存在这样的函数依赖$X \to A$及$Z \subset X$，使得：

$$F^+ = (F - \{X \to A\} \cup \{Z \to A\})^+ ;$$

(3) F中不存在这样的函数依赖$X \to A$：使得

$$F^+ = (F - \{X \to A\})^+$$

则称F为最小函数依赖集或最小覆盖

4.2 数据依赖

最小函数依赖集

- 定理4.2：每一个函数依赖集F均等价于一个最小函数依赖集F_{min}
- 证：这是一个构造性的证明，我们分三步对F进行“极小化处理”，找出F的一个最小依赖集来。
  1. 逐一检查F中各函数依赖$FD_i: X \to Y$，若$Y = A_1A_2 \dots A_k, k > 2$，则用$\{X \to A_j | j = 1,2, \dots, k\}$来取代$X \to Y$
  2. 逐一取出F中各函数依赖$FD_i: X \to A$，设$X = B_1B_2 \dots B_m$,逐一考查$B_i (i = 1,2, \dots, m)$,若$A \in (X - B_i)^+_F$，则以$X - B_i$取代X
  3. 逐一检查F中各函数依赖$FD_i: X \to A$，令$G = F - \{X \to A\}$，若$A \in X^+_G$，则从F中去掉此函数依赖

  注：因为F与G等价的充要条件是$A \in X^+_G$

  因为F与$F - \{X \to A\} \cup \{Z \to A\}$ 等价的充要条件是$A \in Z^+$ 其中$Z = X - B_i$
- 最后剩下的F就一定是极小依赖集,并且与原来的F等价，因为，我们对F的每一次‘改造’都保证了改造前后的两个函数依赖集等价
- 应当指出，F的最小依赖集F_{min}不一定是唯一的，它和我们对各函数依赖FD_i及$X \to A$中X各属性的处置顺序有关

4.2 数据依赖

➢最小函数依赖集

- 例4.4：$R(A,B,C,D,E,H,I)$，$F=\{A\to BE, AH\to D, B\to C, BD\to E, C\to D, H\to I, I\to H, H\to BE\}$,试求$F$的最小依赖集$F_{min}$。
- 解：（1）右部拆成单属性

$$F=\{A\to B, A\to E, AH\to D, B\to C, BD\to E, C\to D, H\to I, I\to H, H\to B, H\to E\},$$

（2）考察左部不是单属性的函数依赖，消除多余属性

$AH\to D$ ∵ $((AH)-H)_F{}^+ = ABECD, D\in((AH)-H)_F{}^+$

∴ 以$A\to D$取代$AH\to D$

$BD\to E$ ∵ $((BD)-D)_F{}^+ = BCDE, E\in((BD)-D)_F{}^+$

∴ 以$B\to E$取代$BD\to E$

$$F=\{A\to B,\ A\to E, A\to D,\ B\to C,\ B\to E,\ C\to D,\ H\to I, I\to H,\ H\to B,\ H\to E\}$$

4.2 数据依赖

➢最小函数依赖集

$$F=\{A\to B,\ A\to E,A\to D,\ B\to C,\ B\to E,\ C\to D,\ H\to I,I\to H,\ H\to B,\ H\to E\}$$

（3）消除多余的函数依赖

$A\to B$ ∵ $AG^+=AED, B\notin A_G{}^+(G=F-\{A\to B\})$ ∴ 保留该函数依赖

$A\to E$ ∵ $AG^+=ABCDE, E\in AG^+(G=F-\{A\to E\})$ ∴ 不保留该函数依赖

$A\to D$ ∵ $(A)_G{}^+=ABCDE, D\in (A)_G{}^+(G=F-\{A\to D\})$ ∴ 不保留该函数依赖

$B\to C$ ∵ $BG^+=B,C\notin BG^+(G=F-\{B\to C\})$ ∴ 保留该函数依赖

$B\to E$ ∵ $(B)_G{}^+=BCD, E\notin (B)_G{}^+(G=F-\{B\to E\})$ ∴ 保留该函数依赖

$C\to D$ ∵ $CG^+=AE, D\notin CG^+(G=F-\{C\to D\})$ ∴ 保留该函数依赖(此处错误，闭包应为C)

$H\to I$ ∵ $HG^+=HBECD, I\notin HG^+(G=F-\{H\to I\})$ ∴ 保留该函数依赖

$I\to H$ ∵ $IG^+=I,H\notin IG^+(G=F-\{I\to H\})$ ∴ 保留该函数依赖

$H\to B$ ∵ $HG^+=HIE, B\notin HG^+(G=F-\{H\to B\})$ ∴ 保留该函数依赖

$H\to E$ ∵ $HG^+=HBCDE, E\in HG^+(G=F-\{H\to E\})$ ∴ 不保留该函数依赖

最后剩下的F是最小函数依赖集，

$$F_{min}=\{A\to B,\ B\to C,\ B\to E, C\to D,\ H\to I,\ I\to H,\ H\to B\}$$

4.2 数据依赖

最小函数依赖集

- 应当指出，F的最小依赖集F_{min}不一定是唯一的，它和我们对各函数依赖FD_i及$X \rightarrow A$中X各属性的处置顺序有关

$$F = \{A \rightarrow B,\ B \rightarrow A,\ B \rightarrow C,\ A \rightarrow C,\ C \rightarrow A\}$$

4.2 数据依赖

➢多值依赖

- 除了函数依赖以外，关系的属性间还存在其他一些数据依赖关系，多值依赖(multivalued dependency,MVD) 就是其中之一
- *例1 ：学校中某一门课程由多个教员讲授，他们使用相同的一套参考书。每个教员可以讲授多门课程，每种参考书可以供多门课程使用。我们可以用一个非规范化的关系来表示教员T, 课程C 和参考书B 之间的关系：*

课程C	教员T	参考书B
物理	李 勇	普通物理学
	王 军	光学原理
		物理习题集
数学	刘 斌	数学分析
	张 平	微分方程
		高等代数
计算数学	王 娟	数学分析
	周 峰	……

4.2 数据依赖

➢多值依赖

•把这张表变成一张规范化的二维表，就成为 :Teaching

课程C	教员T	参考书B
物 理	李 勇	普通物理学
物 理	李 勇	光学原理
物 理	李 勇	物理习题集
物 理	王 军	普通物理学
物 理	王 军	光学原理
物 理	王 军	物理习题集
数 学	刘 斌	数学分析
数 学	刘 斌	微分方程
数 学	刘 斌	高等代数
数 学	张 平	数学分析
数 学	张 平	微分方程
数 学	张 平	高等代数

4.2 数据依赖

对于W的每一个值， S有一个完整的集合与它对应，不论S取什么值，所以W S

➢MVD 举例

•关系模式 WSC(W,S,C) 中， W 表示仓库， S 表示保管员， C 表示商品。假设每个仓库有若干个保管员，有若干种商品。每个保管员保管所在仓库的所有商品，每种商品被所有保管员保管

W	S	C
W1	S1	C1
W1	S1	C2
W1	S1	C3
W1	S2	C1
W1	S2	C2
W1	S2	C3
W2	S3	C4
W2	S3	C5
W2	S4	C4
W2	S4	C5

4.2 数据依赖

MVD举例

- 例：设关系模式 $R(A,B,C)$ 上有一个MVD A B，如果 R 中已有 $(a,b1,c1),(a,b2,c2)$，$(a,b3,c3)$，那么这些关系中至少还应该存在哪些元组?

至少还存在 **6** 个元组：

(a,b1,c2)(a,b2,c1)(a,b1,c3)

(a,b3,c1)(a,b2,c3)(a,b3,c2)

4.2 数据依赖

多值依赖与函数依赖

- 有效性范围
  - $X \rightarrow\rightarrow Y$的有效性与属性集范围有关
  - $X \rightarrow\rightarrow Y$在属性集$W$（$XY \quad W \quad U$）上成立，但在$U$上不一定成立
  - $X \rightarrow\rightarrow Y$在$U$上成立 $\Rightarrow X \rightarrow\rightarrow Y$在属性集$W$（$XY \quad W \quad U$）上成立

4.3 关系模式分解

➢关系模式分解

- 我们可以通过把一个“大的”关系模式分解成多个“小的”关系模式，以解决插入异常、删除异常和更新异常等问题
- 实际上，关系模式的分解，不仅仅是属性集合的分解，也是对关系模式上的函数依赖集，以及关系模式的当前值的分解的具体表现
- 关系模式的分解并不是随意的，必须保证原来的关系模式的语义性质和信息不被丢失，即要求分解应当既“无损联接”又“保持函数依赖”

4.3 关系模式分解

SCT

S#	C#	Grade	Teacher	Dept
02055019	CS - 01	89	刘朝	计算机
02055019	CS - 03	86	张莹	计算机
02055037	CS - 01	68	刘朝	计算机
02055066	CS - 02	92	林琳	计算机
02055066	CS - 03	73	张莹	计算机
…	…	…	…	…

SC

S#	C#	Grade
02055019	CS - 01	89
02055019	CS - 03	86
02055037	CS - 01	68
02055066	CS - 02	92
02055066	CS - 03	73
020550	…	…

CT

C#	Teacher
CS - 01	刘朝
CS - 02	林琳
CS - 03	张莹
…	…

TD

Teacher	Dept
刘朝	计算机
林琳	计算机
张莹	计算机
…	…

4.3 关系模式分解

SCT

S#	C#	Grade	Teacher	Dept
02055019	CS—01	89	刘朝	计算机
02055019	CS—03	86	张莹	计算机
02055037	CS—01	68	刘朝	计算机
02055066	CS—02	92	林琳	计算机
02055066	CS—03	73	张莹	计算机
…	…	…	…	…

SC

S#	C#	Grade
02055019	CS—01	89
02055019	CS—03	86
02055037	CS—01	68
02055066	CS—02	92
02055066	CS—03	73
…	…	…

CT

C#	Teacher
CS—01	刘朝
CS—02	林琳
CS—03	张莹
…	…

TD

Teacher	Dept
刘朝	计算机
林琳	计算机
张莹	计算机
…	…

操作	分解前（冗余）	分解后（无冗余）
存储教师系别信息	每个选课学生重复存储	每个教师仅存储 1 次
更新教师系别	需修改所有相关学生选课记录（O(N) 次）	仅需修改 **TD** 表（O(1) 次）
存储空间	冗余度高（空间复杂度 O(N)）	冗余消除（空间复杂度 O(M), M ≪ N）

4.3 关系模式分解

泛关系假设

- 假设已知一个数据库$\sigma = <r_1, \ldots, r_k>$，存在着仅由单个关系模式$R$上的对应实例$r$与之等价。
- 或者说，对于一个关系数据库，可以将所有属性放入一张（非常宽的）大表中，其同时可以被分解为多个小表。
- 再或者，先存在一个r的情况下，再去谈论分解。

4.3 关系模式分解

关系模式分解的定义

- 定义4.14：设$R(U)$为关系模式，则称:$\rho = \{R_1(U_1), R_2(U_2), ..., R_k(Uk)\}$ (其中$U = \cup_{i=1}^{k} U_i$,且对于任意的$1 \leq i, j \leq k$没有$U_i \subseteq U_j$)为R的一个分解。
- 定义4.15：函数依赖集F在属性集U_i ($\subseteq U$)上的投影定义为:

$$\prod_{U_i}(F) = \{X \rightarrow Y | X \rightarrow Y \in F^+ \wedge XY \subseteq U_i\}$$

- R称为泛关系模式，R对应的当前值称为泛关系
- 数据库对应的当前值σ称为数据库实例，它是由数据库模式中的每一个关系模式的当前值组成，即

$$\sigma = < r_1, r_2, ..., r_k >$$

4.3 关系模式分解

- 泛关系模式

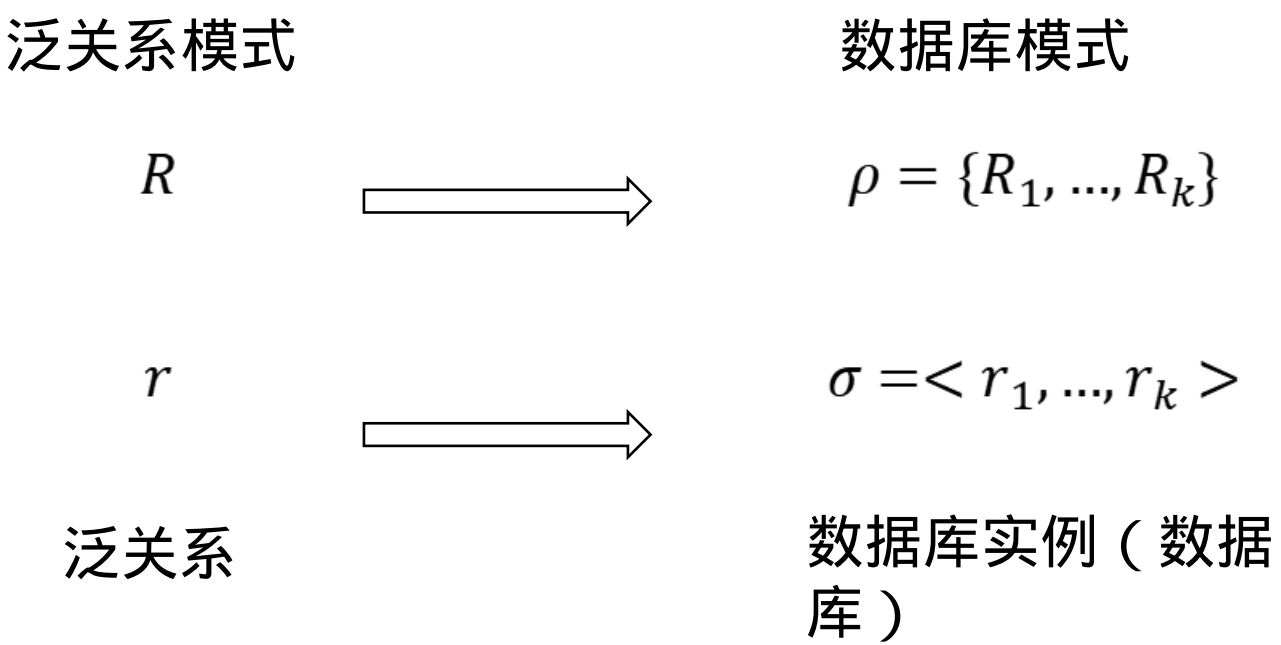


图 4.5　模式分解示意图

4.3 关系模式分解

➢泛关系模式

泛关系模式 R	⟹	数据库模式 $\rho = \{R_1, \ldots, R_k\}$
r 泛关系	⟹	$\sigma = <r_1, r_2, \ldots, r_k>$ 数据库实例(数据库)
函数依赖集F	……	$\{F_1, \ldots, F_k\}$

这里有两个问题：

①σ和r是否等价？　用“无损分解”表示

②$\{F_1, \cdots, F_k\}$与F是否等价？用“保持依赖”表示

4.3 关系模式分解

无损联接分解

- 定义4.16：设R是一个关系模式，F是R上的一个FD集，$\rho = \{R_1, ..., R_k\}$为R的一个分解,如果对R中满足F的每一个关系r，都有

$$r = \pi_{R_1}(r) \bowtie \pi_{R_2}(r) \cdots \bowtie \pi_{R_k}(r)$$

- 则称分解ρ相对于R是“无损联接分解”(lossless join decomposition)，简称“无损分解”，否则，称ρ为”有损联接分解” (loss join decomposition),简称“有损分解”。

4.3 关系模式分解

- 无损联接分解

r	A	B	C
	1	1	1
	1	2	1

(a)

r_1	A	B
	1	1
	1	2

(b)

r_2	A
	1
	1

(c)

图 未丢失信息的分解

	A	B	C
r	1	1	4
	1	2	3

(a)

r_1	A	B
	1	1
	1	2

(b)

r_2	A	C
	1	4
	1	3

(c)

图 丢失信息的分解

r	$r_1 \bowtie r_2$ A	B
	1	1
	1	1
	1	2
	1	2

(d)

4.3 关系模式分解

投影连接运算

- 定义4.17：设$\rho = \{R_1, ..., R_k\}$为R的一个分解，r是R上的任意一个具体关系，则r对于ρ的投影连接运算定义为：

$$m_\rho(r) = \bowtie_{i=1}^{k} \pi_{U_i}(r)$$

- 引理4.4：设$\rho = \{R_1, ..., R_k\}$是R的一个分解，r是R的任一关系，$r_i = \pi_{R_i}(r)(1 \leq i \leq k)$，那么有下列性质：
  - $r \subseteq m_\rho(r)$；
  - 若$s = m_\rho(r)$，则$\pi_{R_i}(r) = r_i$；
  - $m_\rho\big(m_\rho(r)\big) = m_\rho(r)$，这个性质称为幂等性(Idempotent)

4.3 关系模式分解

➢投影连接运算

➢**证明:**

(1) 任取$t \in r$,则$t[R_i] \in r_i\ (i = 1,2,\ldots,k)$。由于$t[R_1],\ldots,t[R_k]$本来取自$t$在$R_i$上的投影，因此在进行连接时，必然相互匹配，拼接成元组t，故$t \in m_\rho\ (r)$，即 $r \subseteq m_\rho\ (r)$

(2) 由(1)可知，$r \subseteq s$, 因此$\prod_{R_i}(r) \subseteq \prod_{R_i}(s)$或$r_i \in \prod_{R_i}(s)$。现只须证$\prod_{R_i}(s) \subseteq r_i$。为此任取$u \in \prod_{R_i}(s)$，则必有$t_s \in s$,使$t_s[R_i] = u$，由$s$的构造方式可知$t_s[R_i] \in r_i(i = 1,\ldots,k)$,即$u \in r_i$,因此有$\prod_{R_i}(s) \subseteq r_i$

(3) 由$\prod_{R_i}(s) = r_i$，可知:

$$m_\rho\left(m_\rho(r)\right) = m_\rho(s) = \bowtie_{i=1}^{k} \prod_{R_i}(s) = \bowtie_{i=1}^{k}(r_i) = m_\rho(r)$$

(证毕)

4.3 关系模式分解

无损联接分解

- 由引理4.4可得到下面的结论：
  - 如果 $r \neq m_\rho(r)$，则ρ不是无损分解。
  - 由引理4.4(1)可知， r经过分解后再连接，如果ρ不是无损分解，则所得到的结果的元组数总比原来的r多，即所谓“元组增加，信息丢失(有损)！”
  - 由引理4.4(2)可知，对r进行$m_\rho(r)$变换后得到的s可满足条件： $\prod_{R_i}(s) = r_i$
  - 但注意：如果r_i不是r的投影，而是独立的关系，则一般而言，连接后的关系s不再满足条件$\prod_{R_i}(s) = r_i$，而是$\prod_{R_i}(s) \subseteq r_i$

4.3 关系模式分解

无损联接分解

- 例：设关系模式$R(ABC)$分解成$\rho = \{AB, BC\}$，(a)和(b)分别是模式AB和BC上的值r_1和r_2，(c)是$r_1 \bowtie r_2$的值。显然$\pi_{BC}(r_1 \bowtie r_2) \neq r_2$。这里$r_2$中元组$(b_2, c_2)$就是一个悬挂元组，由于它的存在，使得$r_1$和$r_2$不存在泛关系$r$

r_1	A	B
	a_1	b_1

（a）关系 r_1

r_2	B	C
	b_1	c_1
	b_2	c_2

（b）关系 r_2

$r_1 \bowtie r_2$	A	B	C
	a_1	b_1	c_1

（c）$r_1 \bowtie r_2$

4.3 关系模式分解

➢无损分解的判别方法

1. 构造一张k行n列的表格，每列对应一个属性A_j（$1 \le j \le n$），每行对应一个模式R_i（$1 \le i \le k$）。如果A_j在R_i中，那么在表格的第i行第j列处填上符号a_j，否则填上b_{ij}
2. 把表格看成模式R的一个关系，反复检查F中每个FD在表格中是否成立，若不成立，则修改表格中的值。修改方法如下：对于F中一个FD $X \to Y$，如果表格中有两行在X值上相等，在Y值上不相等，那么把这两行在Y值上也改成相等的值。如果Y值中有一个是a_j，那么另一个也改成a_j；如果没有a_j，那么用其中一个b_{ij}替换另一个值（尽量把下标ij改成较小的数）。一直到表格不能修改为止（这个过程称为chase过程)
3. 若修改的最后一张表格中有一行是全a，即$a_1a_2...an$，那么称ρ相对于F是无损分解，否则称损失分解

4.3 关系模式分解

无损分解的判别算法

- 算法4.3: 检验一个分解是否无损连接分解
  - 输入：关系模式$R(A_1, ..., A_n)$； R上的函数依赖集F；
    R的一个分解$\rho = \{R_1, R_2, ..., R_k\}$；
  - 输出： ρ是否无损连接分解
  - 方法：(1) 建立一个n列、k行的符号表M：

…	…	…	…
…	…	…	…
…	…	…	…
	A_1	A_2	A_j
…	…	…	…

4.3 关系模式分解

➢无损分解的判别算法

- 符号表 M 各元素的值由下面的规则确定：

$$M[i,j]=\begin{cases} a_j \text{若} A_j \in U_i \\ bij \text{若} A_j \notin U_i \end{cases}$$

- 用F中的每一函数依赖$X \to Y$对M反复进行下列检查和处理：检查X中的属性所对应的列，找出在X上取值相等的行，如果找到两个(或多个)行在X上取值相等，就将对应行中Y中属性所对应的符号改为一致，即如果其中之一为a_j，则将其他符号也改为a_j；如果全部符号都是“b”符号，则将它们改为同样的“b”符号
- 如此进行下去，直到发现M中某一行变为：$a_1,a_2,\ldots\ldots,a_n$，则说明ρ是无损连接分解；
- 否则，一直进行到M不再改变为止，则说明ρ不是无损连接分解

4.3 关系模式分解

无损分解的判别算法

- 例: $R(A, B, C, D, E)$

$$F = \{A \to C,\ B \to C,\ C \to D,\ DE \to C,\ CE \to A\}$$

- 判断:

$$\rho = \{R_1(A, D), R_2(A, B), R_3(B, E), R_4(C, D, E), R_5(A, E)\}$$

是否是无损连接的分解?

4.3 关系模式分解

➢无损分解的判别算法

$R(A,B,C,D,E)\ F=\{A\to C,\ B\to C,\ C\to D,\ DE\to C,\ CE\to A\}$

$\rho=\{R_1(A,D),R_2(A,B),R_3(B,E),R_4(C,D,E),R_5(A,E)\}$

1、构造初始表格

	A	B	C	D	E
AD	a1	b12	b13	a4	b15
AB	a1	a2	b13	a4	b25
BE	a1	a2	a3	a4	a5
CDE	a1	b42	a3	a4	a5
AE	a1	b52	a3	a4	a5

2、处理表格

$A\to C$：将b23，b53改为b13

$B\to C$：将b33改为b13

$C\to D$：将b24，b34，b54改为a4

$DE\to C$：将第3行和第5行的C改为a3

$CE\to A$：将第3行和第4行的A改为a1

因此是无损连接的分解

4.3 关系模式分解

无损分解的判别算法

- 对于只有2个关系模式的分解可以通过下面的定理判断其是否为无损联接分解
- 定理4.4：设$\rho = \{R_1, R_2\}$是关系模式R的一个分解，F是R上成立的FD集，那么分解ρ相对于F是无损分解的充分必要条件是$(R_1 \cap R_2) \rightarrow (R_1 - R_2)$或$(R_1 \cap R_2) \rightarrow (R_2 - R_1)$
- 定理4.5：如果FD：$X \rightarrow Y$在模式R上成立，且$X \cap Y = \varphi$，则$\rho = \{U - Y, XY\}$是R的无损分解

4.3 关系模式分解

保持函数依赖的分解

- 保持关系模式分解等价的另一个重要条件是关系模式的函数依赖集在分解后仍在数据库模式中保持不变
- 即关系模式R到$\rho = \{R_1, R_2, ..., R_k\}$的分解，应使函数依赖集$F$,被$F$在这些$R_i$上的投影蕴涵

4.3 关系模式分解

保持函数依赖的分解

- 定义4.18：设$\rho = \{R_1, R_2, ..., R_k\}$是$R$的一个分解，$F$是$R$上的$FD$集，如果有：

$$\cup_{\pi_{R_i}}(F) = \pi_{R_1}(F) \cup \pi_{R_2}(F) ... \pi_{R_k}(F) \vDash F$$

则称分解ρ保持函数依赖集F

$$\prod_{U_i}(F) = \{X \to Y \in F^+ \wedge XY \subseteq U_i\}$$

4.3 关系模式分解

保持函数依赖的分解

- 下面介绍检验分解是否保持函数依赖的算法
- 检验分解是否保持函数依赖实质上就是检验$\cup_{i=1}^{k}\pi_{U_i}(F)$是否覆盖$F$
- 算法4.4: 检验分解是否保持函数依赖。
  - 输入： $\rho=\{R_1,R_2,...,R_k\}$， 函数依赖集F
  - 输出： ρ是否保持F
  - 方法：令$G=\cup_{i=1}^{k}\pi_{U_i}(F)$
  - 为检验G是否覆盖F， 可对F中的每一函数依赖$X\rightarrow Y$进行下列检验：
  - 首先计算X_G^+， 然后检查Y是否被包含在X_G^+中

4.3 关系模式分解

保持函数依赖的分解：举例

- 设关系模式$R = \{CITY, ST, ZIP\}$满足函数依赖：

  $(CITY, ST) \rightarrow ZIP$

  $ZIP \rightarrow CITY$

  分解成$\rho = \{R_1(ST, ZIP), R_2(CITY, ZIP)\}$

- 检查该分解是否具有保持函数依赖性：

  $\because \prod_{R_1} = \emptyset$（平凡的函数依赖）

  $\prod_{R_2} = \{ZIP \rightarrow CITY\}$

  $\prod_{R_1} \cup \prod_{R_2} = \{ZIP \rightarrow CITY\}$

  $\therefore$ ρ不具有保持函数依赖性

4.3 关系模式分解

➢模式分解与模式等价问题

- 本节讨论的关系模式分解的两个特性实际上涉及到两个数据库模式的等价问题，这种等价包括数据等价和依赖等价两个方面
- 数据等价是指两个数据库实例应表示同样的信息内容，用“无损分解”衡量
- 如果是无损分解，那么对泛关系反复的投影和联接都不会丢失信息
- 依赖等价是指两个数据库模式应有相同的依赖集闭包
- 在依赖集闭包相等情况下，数据的语义是不会出差错的。
- 违反数据等价或依赖等价的分解很难说是一个好的模式设计

4.3 关系模式分解

模式分解与模式等价问题

- *例4.7：关系模式R（ABC），$\rho = \{AB, AC\}$ 是R的一个分解*
- 试分析分别在$F_1 = \{A \to B\}$，$F_2 = \{A \to C, B \to C\}$，$F_3 = \{B \to A\}$，$F_4 = \{C \to B, B \to A\}$情况下，检验$\rho$是否具有无损分解和保持$FD$的分解特性
- 解：
  ① 相对于$F_1 = \{A \to B\}$，分解ρ是无损分解且保持FD的分解
  ② 相对于$F_2 = \{A \to C, B \to C\}$，分解$\rho$是无损分解，但不保持$FD$集。因为$B \to C$丢失了
  ③ 相对于$F_3 = \{B \to A\}$，分解ρ是损失分解（见定理4.5——(R_1交R_2)->(R_1差R_2)）但保持FD集的分解
  ④ 相对于$F_4 = \{C \to B, B \to A\}$，分解$\rho$是损失分解且不保持$FD$集的分解(丢失了$C \to B$)

4.4 关系模式的规范化

➢关系模式的规范化

- 为了使数据库设计的理论和方法走向完备，人们研究了关系规范化理论，以指导我们设计规范化的数据库模式
- 在进行关系模式设计时，首先面临的一个问题是如何评价一个关系模式的“好”与“坏”，即用什么标准衡量？
- 这个标准就是模式的范式 (Normal Forms ，简记为 NF)
- 而使得关系模式达到一定范式要求，就是关系模式的规范化

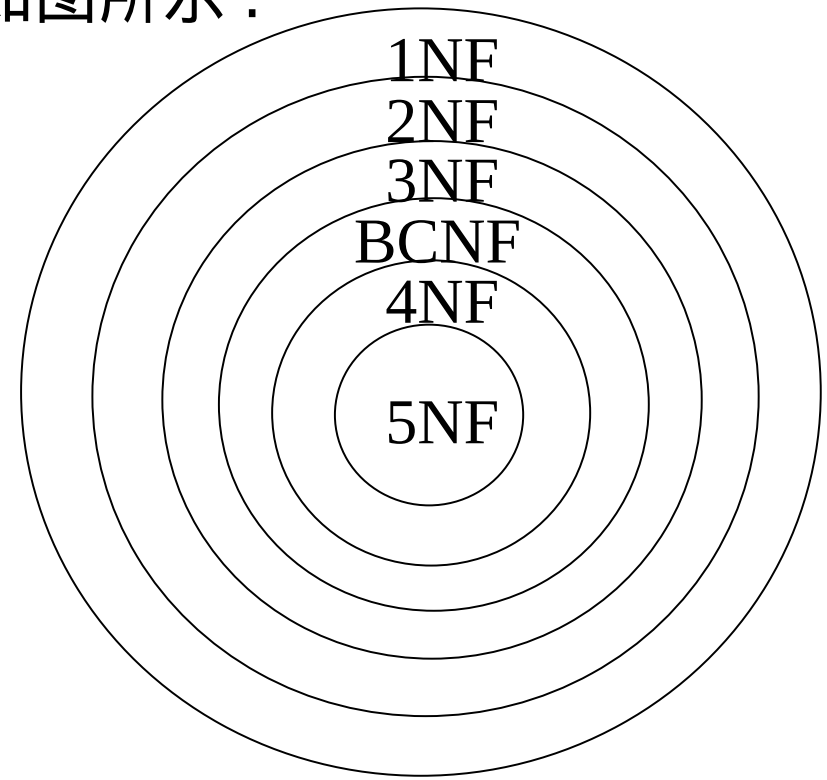
4.4 关系模式的规范化

范式

- 关系数据库中的关系模式要满足一定的规范化要求，满足不同程度规范化要求的关系模式的类,称为不同的范式(Normal Forms，简记为NF)
- 范式的种类与数据依赖有着直接的联系，基于FD的范式有1NF、2NF、3NF、BCNF等多种
- 满足最低要求的关系模式称为第一范式,简称1NF。在第一范式中满足进一步要求的为第二范式，其余以此类推
- R为第几范式就写成$R \in x$NF

4.4 关系模式的规范化

- 各种范式之间的关系
  - 按属性间依赖情况来区分，关系规范化的程度为 1NF,2NF,3NF,BCNF,4NF 和 5NF 等
  - 各种范式之间的关系如图所示：



4.4 关系模式的规范化

- 规范化的过程
  - 1NF 是关系模式的基础； 2NF 已成为历史，一般不再提及；在数据库设计中最常用的是 3NF 和 BCNF
  - 为了叙述的方便，我们按照 1NF 、 2NF 、 3NF 、 BCNF 顺序来介绍

4.4 关系模式的规范化

➢第一范式（ 1NF ）

- 定义 4.19 ：如果关系模式 R 的每个关系 r 的属性值都是不可分的原子值，则称 R 是第一范式（ first normal form ，简记为 1NF ）的模式
- 满足 1NF 的关系称为规范化的关系，否则称为非规范化的关系
- 关系数据库研究的关系都是规范化的关系。
- 例如关系模式 R （ NAME ， ADDRESS ， PHONE ），如果一个人有两个电话号码（ PHONE ），那么在关系中至少要出现两个元组，以便存储这两个号码
- 1NF 是关系模式应具备的最起码的条件

4.4 关系模式的规范化

➢第一范式（ 1NF ）

- 分量是否需要再分，与具体应用有关，如果用到值的一部分，则需要进一步分割，例如：

姓名	生日
王军	68.7.10
张立	69.7.10
李明	80.3.28

姓名	年	月日
王军	68	7.10
张立	69	7.10
李明	80	3.28

- 如果只是查询出生日期，则它满足 1NF
- 如果查询两人生日是否相同，则只比较月、日，需要将生日分解，就不满足 1NF
- 如果比较两人的生肖呢？

4.4 关系模式的规范化

键(Key)

- 定义4.20: 设K为$R<U,F>$中的属性或属性组合，若$K \to U \in F^+$，则称K为R的一个超键
- 定义4.21: 如果一个超键K的任何真子集都不再是超键，则称K为R的一个候选键(Candidate key)
- 候选键有时也简称为键
- 若关系模式中有多个候选键，则选定其中的一个为主键(Primary key)
- 定义4.22: 包含在任何一个候选键中的属性，称为主属性(Prime attribute)或键属性(Key attribute)。不包含在任何候选键中的属性，则称为非主属性(Nonprime attribute)或非键属性(Non-key attribute)

4.4 关系模式的规范化

➢键 (Key)

- 最简单的情况，单个属性是候选键。最极端的情况下，整个属性组是候选键，并称为全键 (All-key)

- 例：关系模式 R(P,W,A), 属性 P 表示演奏者， W 表示作品 ,A 表示听众。假设一个演奏者可以演奏多个作品，某一作品可被多个演奏者演奏。听众也可以欣赏不同演奏者的不同作品，这个关系模式的候选键为 (P,W,A) ，即 All-key

4.4 关系模式的规范化

➢求关系模式上全部候选键

- 求关系模式上的全部候选键的方法
- 输入：关系模式 R(U ， F) ，其中 F 已化简为最小覆盖
- 输出： R 上的所有候选键
- 步骤 (1) ：将 R 的全部属性分为四类，分别是
  - C1 ：不在任何函数依赖中出现的属性；
  - C2 ：仅在函数依赖决定子中出现的属性；
  - C3 ：仅在函数依赖右部出现的属性；
  - C4 ：在函数依赖左部右部均有出现的属性。
  - 注意，任何候选键中必然会包括 C1 、 C2 中的属性

4.4 关系模式的规范化

求关系模式上全部候选键

- 步骤(2)：若$(C1 \cup C2)^+ = U$ 或者 $C4 = \emptyset$，则C1∪C2即为唯一的候选键；否则，逐一将C4中的属性加入C1∪C2并计算其闭包，若其闭包为U，则C1∪C2与该属性构成关系上的一个候选键，重复该过程直至找出所有的候选键

4.4 关系模式的规范化

求关系模式上全部候选键

- *例4.8：求R(A，B，C，D)，F={A→B，B→C，BD→A}的所有候选键：*
- 解：C1 = ∅，C2 = {D}，C3 = {C}，C4 = {A，B}，(C1∪C2)+ = {D}，所以C1∪C2不是候选键，依次将C4中的A、B加入C1∪C2中计算其闭包可得：(AD)+ = ADBC，(BD)+ = BDAC，所以可知R的候选键为AD和BD

4.4 关系模式的规范化

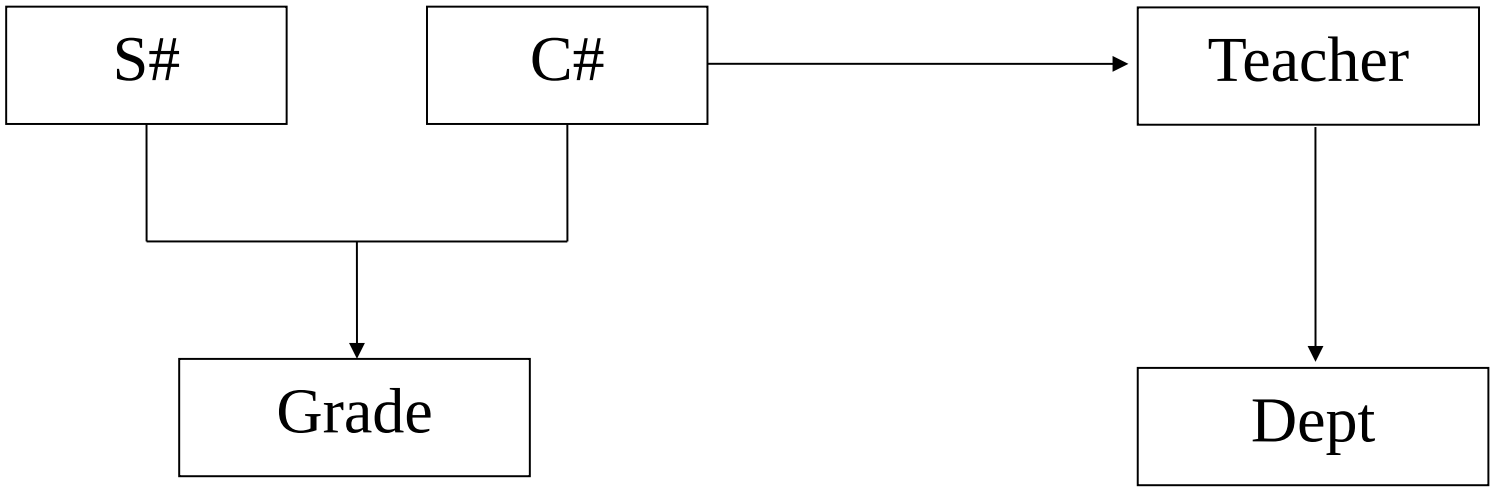
第二范式（2NF）

- 定义4.23：如果关系模式R中的所有非主属性都完全函数依赖于所有CK，则称R满足2NF，表示为R∈2NF
- 2NF是在1NF的基础上消除非主属性对于所有候选键的部分函数依赖
- 如果数据库模式中每个关系模式都是2NF，则称数据库模式为2NF的数据库模式

4.4 关系模式的规范化

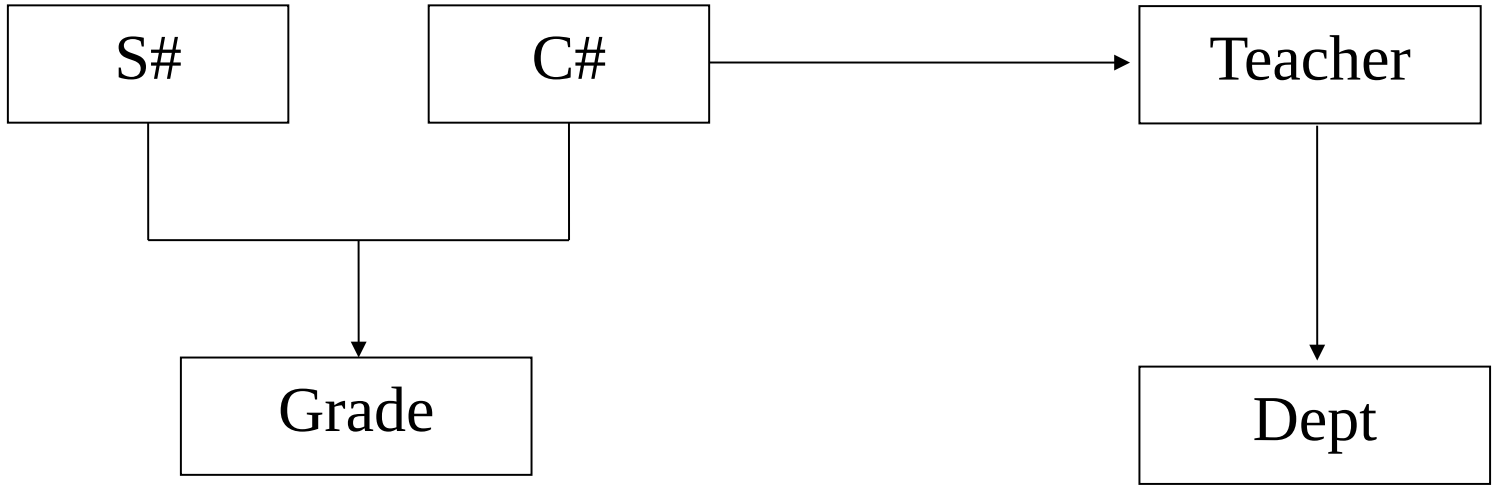
➢第二范式（ 2NF ）

•例 4.14：设关系模式 R（ S# ，C# ，Grade ，Teacher ，Dept ）的属性分别表示学生学号、选修课程的编号、成绩、任课教师姓名和所在系等意义。（ S# ，C# ）是 R 的候选键



4.4 关系模式的规范化

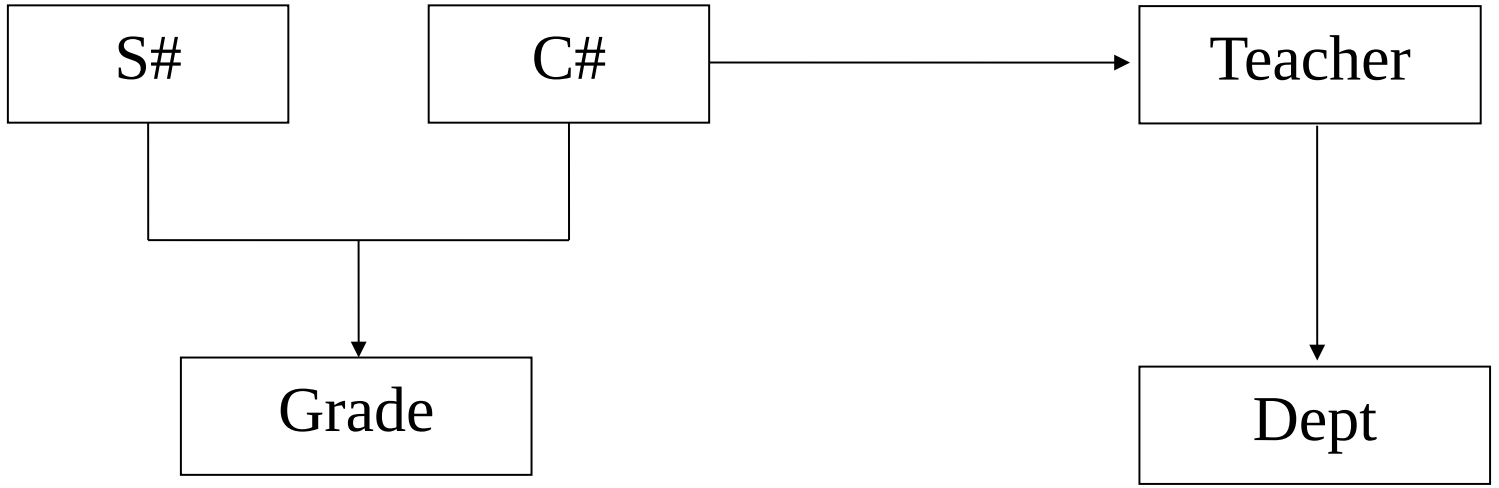
➢第二范式（ **2NF** ）



•R 上有两个 FD ：（ S# ， C# ）→（ Teacher ， Dept ）和 C#→（ Teacher ， Dept ），由于前一个 FD 是部分依赖，因此 R 不是 2NF 模式。此时 R 的关系就会出现冗余和异常现象。譬如某一门课程有 100 个学生选修，那么在关系中就会存在 100 个元组，因而教师的姓名和所在系就会重复 100 次。

4.4 关系模式的规范化

➢第二范式（ **2NF** ）



•如果把 R 分解成 R_1（ C# ，Teacher ，Dept ）和 R_2（ S# ，C# ，Grade ）后，部分依赖（ S# ，C# ）→（ Teacher ，Dept ）就消失了。

R_1 和 R_2 都是 2NF 模式

4.4 关系模式的规范化

➢第三范式（ **3NF** ）

- 定义 4.24 ：如果关系模式 R 中的非主属性既不部分函数依赖也不传递函数依赖于 R 上的所有候选键，则称 R 满足 3NF ，表示为 R∈3NF
- 3NF 是在 2NF 的基础上消除非主属性对于键的传递依赖
- 如果数据库模式中每个关系模式都是 3NF ，则称其为 3NF 的数据库模式

4.4 关系模式的规范化

- 第三范式（ **3NF** ）
  - *例 4.15*　在例 4.14 中，R_2 是 2NF 模式，而且也已是 3NF 模式。但 R_1（ C# ，Teacher ，Dept ）是 2NF 模式，却不是 3NF 模式。如果 R_1 中存在函数依赖 C#→Teacher 和 Teacher→Dept ，那么 C#→Dept 就是一个传递依赖，即 R_1 不是 3NF 模式。此时 R_1 的关系中也会出现冗余和异常操作。譬如一个教师开设五门课程，那么关系中就会出现五个元组，教师的 Dept 就会重复五次。
  - 如果把 R_2 分解成 R_{21}（ Teacher ，Dept ）和 R_{22}（ C# ，Teacher ）后，C#→Dept 就不会出现在 R_{21} 和 R_{22} 中。这样 R_{21} 和 R_{22} 都是 3NF 模式

4.4 关系模式的规范化

➢第三范式（ **3NF** ）

- 推论 1: 如果 R 是 3NF 模式，那么 R 也是 2NF 模式
- 推论 2: 设关系模式 R ，当 R 上每一个 FD: X→A 满足下列三个条件之一：

1. A∈X（即 X→A 是一个平凡的 FD ）；
2. X 是 R 的超键；
3. A 是主属性。

则关系模式 R 就是 3NF 模式。

4.4 关系模式的规范化

- 第三范式（ **3NF** ）

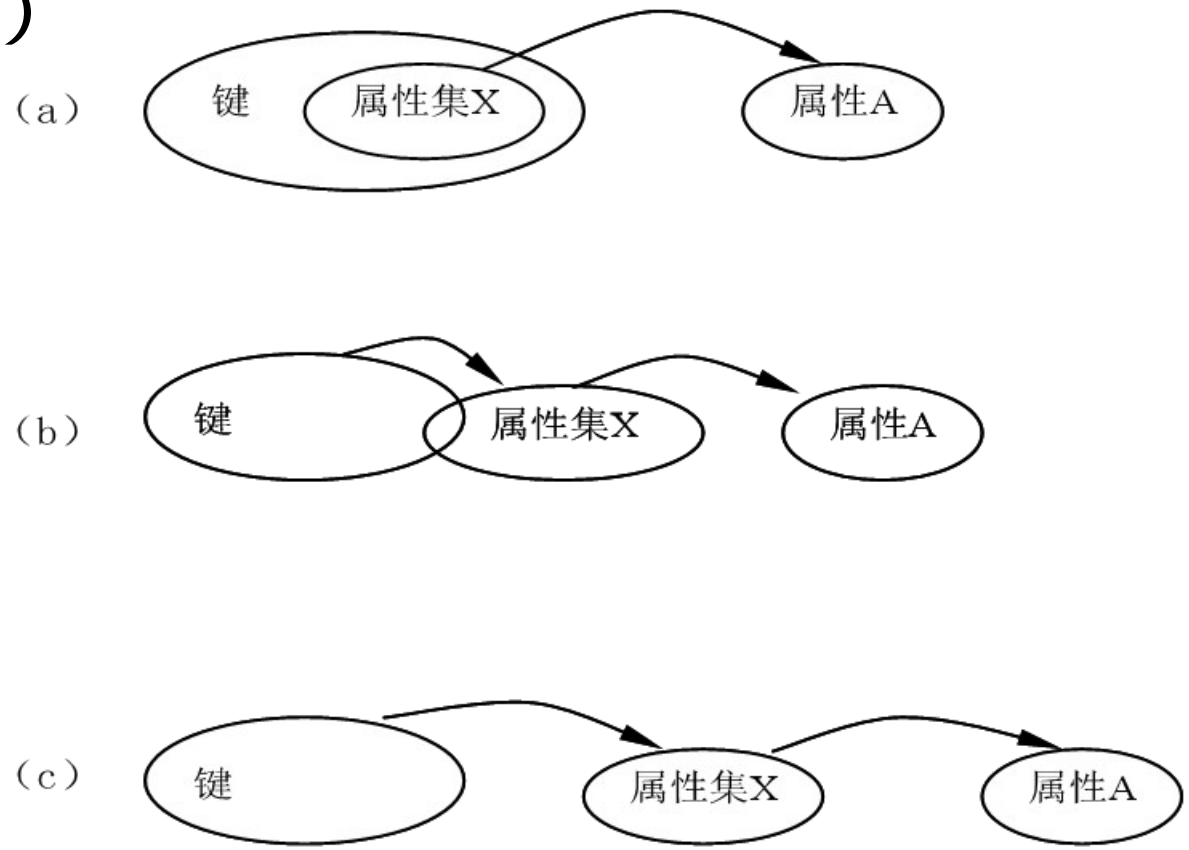


图 6.15　违反 3NF 的传递依赖的三种情况

4.4 关系模式的规范化

➢BCNF（ Boyce–Codd NF ）

- 定义 4.25 ：若对于 R 上的任何非平凡函数依赖 X→Y 都有 X 必包含 R 的某个候选键，则称 R 满足 BCNF ，表示为 R∈BCNF
- 定义 4.25' ：如果关系模式 R 中的每个属性都不传递依赖于 R 的候选键，那么称 R 是 BCNF 的模式

4.4 关系模式的规范化

➢BCNF（ Boyce-Codd NF ）

- 推论 3 ：如果 R 是 BCNF 模式，那么 R 也是 3NF 模式
- BCNF 是在 3NF 的基础上消除主属性对于键的部分依赖和传递依赖
- 如果数据库模式中每个关系模式都是 BCNF ，则称为 BCNF 的数据库模式。

4.4 关系模式的规范化

- **BCNF（ Boyce–Codd NF ）**

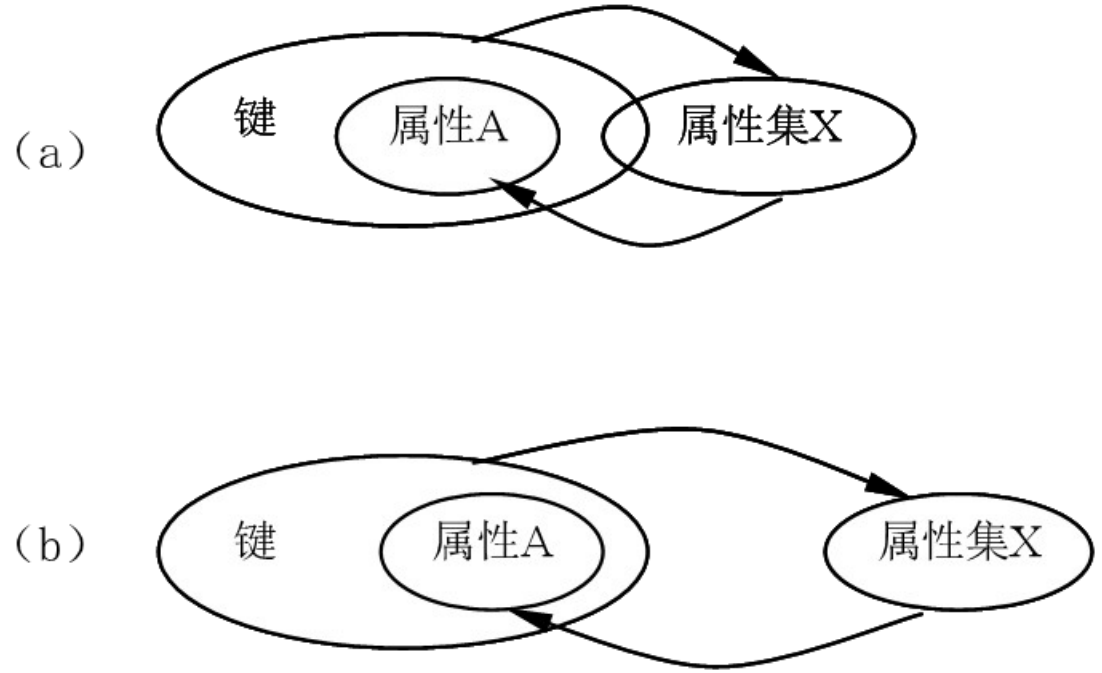


图 6.16　主属性对候选键的传递依赖

4.4 关系模式的规范化

➢BCNF（ Boyce-Codd NF ）

- 上述定义说明，若关系模式 R<U,F> 中的每一个决定因素都包含键，则 R∈BCNF
- 这就是说，在 BCNF 中，除了键决定其它所有属性这样的函数依赖及其所蕴涵的函数依赖外，不再有其它非平凡的函数依赖，特别是不存在以不含键的属性组作为决定子（即左部）的非平凡的函数依赖。
- BCNF 在概念上已足够“单纯”，就函数依赖而言，它已进行了所有必须的分解，并消除了增、删、改异常和数据冗余

4.4 关系模式的规范化

➢BCNF 举例

- *例1 ： 关系模式SJP(S,J,P) 中， S 表示学生，J 表示课程， P 表示名次；每一个学生选修每门课程的成绩有一定的名次，每门课程中每一名次只有一个学生（即没有并列名次），由语义可得到下面的函数依赖：*

$$(S,J)\rightarrow P \quad , \quad (J,P)\rightarrow S$$

- (S,J) 与 (J,P) 都可以作为候选键
- 这两个键各由两个属性组成，而且它们是相交的
- 这个关系模式中显然没有非主属性对键的传递依赖或部分依赖，所以 SJP 是 3NF
- 而且除 (S,J) 与 (J,P) 以外没有其它决定因素，所以 SJP 也是 BCNF

4.4 关系模式的规范化

➢BCNF 举例

- *例2 ：关系模式STJ(S,T,J) 中，S 表示学生，T 表示教师，J 表示课程。每一教师只教一门课。每门课有若干教师，某一学生选定某门课，就对应一个固定的教师。由语义可得到如下的函数依赖：*

$$(S,J)\rightarrow T \ ; \ (S,T)\rightarrow J \ ; \ T\rightarrow J$$

- 这里 (S,J),(S,T) 都是候选键。
- 由于 STJ 中没有非主属性，因此也不会任何非主属性对键传递依赖或部分依赖，所以 STJ 是 3NF
- 但 STJ 不是 BCNF ，因为 T 是决定因素，而 T 不包含键

4.4 关系模式的规范化

➢BCNF 举例

- 例 3 ：关系模式 Teaching(Cname ， Class ， Tname)
- 假定每门课程由多名教师承担多个班级的教学任务，同时规定每名老师仅限承担一门课程的教学任务，但是可以同时给若干个班级带课，于是有(Cname ， Class)→Tname ， Tname→Cname
- Teaching 是 NF

4.4 关系模式的规范化

➢BCNF 举例

- 3NF 的“不彻底”性表现在可能存在主属性对键的部分依赖和传递依赖
- 任何非 BCNF 的关系模式都可以通过分解规范化为 BCNF
- 例如： STJ 可分解为 ST(S,T) 与 TJ （ T,J），并且它们都是 BCNF ，但注意这一分解已不保持函数依赖

4.4 关系模式的规范化

➢ BCNF 举例

- *例4 ：设有关系模式R(S,C,Z) ，其中S 表示街道(street) ， C 表示城市(city) ，Z 表示邮编(zip) ；由语义可以得到以下函数依赖集：*

$$F=\{CS\rightarrow Z\ ,\ Z\rightarrow C\}$$

- CS 是 R 的键
- 在 F 中， Z→C 的左部虽不是超键，但 C 是主属性，在 3NF 中允许这样函数依赖，而 BCNF 中则不允许
- 因此， R 属于 3NF ，但不属于 BCNF
- R 在被更新时仍可能发生异常，例如要插入一邮政编码与某个城市的对应关系，则必须知道该邮政编码所对应的一个街道
- 我们可以将 R 分解为 R1(Z,C) 和 R2(S,Z) ，其中 R1 和 R2 都是 BCNF ，并且这一分解是无损的，但却不保持函数依赖

4.4 关系模式的规范化

➢BCNF 举例

- 一般而言，属于 3NF 但不属于 BCNF 的关系并不多见
- 而且，即使出现这样的关系模式，其引起的更新异常也不是很严重
- 例如，上面例子中的 R(S,C,Z) ，虽然不属于 BCNF ，但基本上是“合理”的关系模式
- 任何关系模式都可以分解为一组 3NF ，且既无损连接，又保持函数依赖
- 若要求分解既保持函数依赖，那么模式分解可以达到 3NF ，但是不一定达到 BCNF

4.4 关系模式的规范化

➢判断范式级别

- 给定关系模式和函数依赖集合，要求判断达到的最高范式。
- 方法：

1.求出给定关系的候选键（可能不止一个）

2.根据键，确定主属性和非主属性。

3.判断是否满足第一范式（看属性的值域是否可以分解）

4.判断是否满足第二范式（非主属性对键的部分函数依赖）

5.判断是否满足第三范式（非主属性对键的传递函数依赖）

6.判断是否满足 BCNF 范式（主属性对键的传递函数依赖）

4.4 关系模式的规范化

➢判断范式级别：举例

- *例1: 已知关系R（A,B,C,D,E）和F={AB→CE,E→AB,C→D}，试判断该关系最高为第几范式？*

解：

1、关系 R 的候选键为： AB 和 E。

因为 :$(AB)^+$={ABCED}

$(E)^+$={EABCD}

AB 可以决定关系的所有属性， E 可以决定关系的所有属性，所以 AB 和 E 是关系的候选键

2、主属性为： A、 B、 E ，非主属性为： C、 D

3、判断是否满足各个范式的要求：

- 因为 R 的所有的属性值域都是不可再分，所以 R 已经在 1NF 中了
- 该关系为 2NF ，因为 C、 D 不存在对任何键的部分依赖

4.4 关系模式的规范化

- 判断范式级别：举例
  - *例2: 已知关系R（A,B,C,D,E,G）和F={A→B,C →DG,AC→E,D→G}，该关系最高为第几范式？*

解：

- 该关系的键为 AC ，因为 $(AC)^+$={ACBDGE}
- 主属性为： A、 C
- 非主属性为： B,D,E,G
- 该关系明显满足 1NF ，但是由于 A→B,C →DG 存在非主属性对键的部分函数依赖，所以该关系不是 2NF
- 所以 R 满足的最高范式是 1NF 。

4.4 关系模式的规范化

- 判断范式级别：举例
  - *例3 ：已知关系R（A,B,C,D ）和F={AB→D,AC →BD,B→C}，试求出其最高范式？*

解：该关系的键是： AB 和 AC 。

- 主属性是 {ABC}, 非主属性是 {D} 。
- D 不部分依赖于任何键，所以该关系在 2NF 中。
- 该关系也在 3NF 中，因为只有一个非主属性，从而不可能存在两个不同非主属性间的传递依赖
- 该关系不在 BCNF 中，因为有 B→C ，而 B 不是 R 的键

4.4 关系模式的规范化

➢分解成 3NF 模式集的算法

- 算法 4.4 ：既无损连接又保持函数依赖的 3NF 分解算法
- 输入：关系模式 $R_R(U_R , F_R)$
- 输出： $\rho = \{ R_0 , R_1 , \ldots , R_k , R_{ck}\}$

4.4 关系模式的规范化

➢分解成 **3NF** 模式集的算法

•步骤：

(1) 首先将 F_R 转换为等价的最小覆盖，记为 F ；

(2) 将不在 F 中出现的属性单独构成关系模式 $R_0(U_0)$ ，其余属性记为 $U = U_R - U_0$ ，　　于是得到第一个分解 $= \{R_0, R\}$ ，其中 R(U ， F) ；

(3) 如果 R 中有某一个函数依赖包含 U 中的全部属性，则算法终止，输出即为 ρ ；

4.4 关系模式的规范化

➢分解成 **3NF** 模式集的算法

•步骤：

(4) 否则，将 F 中的函数依赖按照左部相同原则进行分组，假设共分为 k 组，得到 k 个关系模式 $R_i(U_i, F_i)$，U_i 表示每组函数依赖所涉及的全部属性，F_i 是相应的函数依赖集，将这 k 个关系模式加入 ρ 中得到：$\rho = \{R_0, R_1, \ldots, R_k\}$；

4.4 关系模式的规范化

➢分解成 **3NF** 模式集的算法

•步骤：

(5) 经过上述步骤得到的每个 R_i 均是 3NF ，且因为分组是按照函数依赖进行的，不　　会丢失任何一个函数依赖，所以该过程是保持函数依赖的。为了保证该分解同时是无损连接的，需要再加入一个由任一候选键构成的关系模式 R_{ck} ，如果该关系模式已在前面的某个 R_i 中出现，则可以省略；

(6) 输出 $\rho = \{ R_0 , R_1 , \ldots , R_k , R_{ck} \}$ ，算法结束。

4.4 关系模式的规范化

- 分解成 **3NF** 模式集的算法
  - *例 4.6 ：与学生有关的关系模式 R(S# ， Sname ， Province ， City ， Street ，Zipcode)，各属性分别表示学号、姓名、来自省份、城市、街道及邮政编码*
  - R 上有函数依赖集 F = {S#→Sname ， S#→Province ， S#→City ， S#→Street ， S#→Zipcode，(Province ， City ， Street)→Zipcode，Zipcode→Province ，Zipcode→City}，试将其分解为 3NF 形式且保持无损连接及函数依赖

4.4 关系模式的规范化

➢分解成 **3NF** 模式集的算法

•解：

(1) 首先计算 F 的最小覆盖 F_{min} ，因 S#→Zipcode 冗余，所以：

F_{min} = {S#→Sname ， S#→Province ， S#→City ， S#→Street ， (Province ， City ， Street)→Zipcode， Zipcode→Province ， Zipcode→City} ；

4.4 关系模式的规范化

分解成3NF模式集的算法

- 解：

(2)由于R的所有属性均在F_{min}中出现，所以直接将函数依赖分组构成新的关系模式，ρ={R_1(S#，Sname，Province，City，Street)，R_2(Province，City，Street，Zipcode)，R_3(Zipcode，Province，City)}，因R_3是R_2的子集，故将其略去，得到 ρ={R_1(S#，Sname，Province，City，Street)，R_2(Province，City，Street，Zipcode)}；

4.4 关系模式的规范化

分解成3NF模式集的算法

- 解:

(3)关系R上的候选键只有一个S#，因其也是R_1的子集，所以不再加入ρ中，于是满足要求的最终关系模式集合仍为:

$$\rho = \{R_1(\text{S\#}, \text{Sname}, \text{Province}, \text{City}, \text{Street}),$$

$$R_2(\text{Province}, \text{City}, \text{Street}, \text{Zipcode})\}$$

4.4 关系模式的规范化

分解成BCNF模式集的算法

- 算法4.5：保证无损连接的BCNF分解算法
- 输入：关系模式$R(U, F)$
- 输出：$\rho = \{R_1, ..., R_k\}$
- 步骤：

(1)令$\rho = \{R(U, F)\}$；

(2)如果ρ中的每个子关系模式均是BCNF，则算法终止，输出即为ρ；

4.4 关系模式的规范化

分解成BCNF模式集的算法

(3)否则，ρ中必有某个子关系模式$R_i(U_i, F_i)$ 不满足BCNF，根据BCNF的定义，至少有一个$X \to A \in F^+{}_i$，且X不包含R_i的任一候选键，将R_i分解为两个关系模式R_{i1}、R_{i2}，且$R_{i1} = \{XA，F_{i1}\}$，$R_{i2} = \{U_i - A，F_{i2}\}$，用$R_{i1}$、$R_{i2}$取代$R_i$，返回(2)继续执行

4.4 关系模式的规范化

➢分解成 **BCNF** 模式集的算法：举例

- 已知关系模式 R(CTHRSG), 其中
- C: 表示课程　　T: 表示教师　　H: 表示时间
- R: 表示教室　　S: 表示学生　　G: 表示成绩

键为 HS

- C →T: 每门课程仅有一名教师讲授
- HR →C: 在任一时间，每个教室只能上一门课程
- HT →R: 在一个时间，一个教师只能在一个教室上课
- CS →G: 每个学生的每门课程只有一个成绩
- HS →R: 在一个时间，每个学生只能在一个教室内听课

4.4 关系模式的规范化

➢分解成 BCNF 模式集的算法：举例

- CTHRSG 经过三次分解，分解为 CSG,CT,CHR,CHS 模式。这四个关系模式分别描述了：
  - CSG: 学生的各门课程成绩
  - CT: 各门课的任课教师
  - CHR: 每门课程的上课时间和每个时间的上课教室
  - CHS: 每个学生的上课时间表
- 这些模式都是 BCNF ，而且这些分解是无损分解。
- 但是保持函数依赖吗？

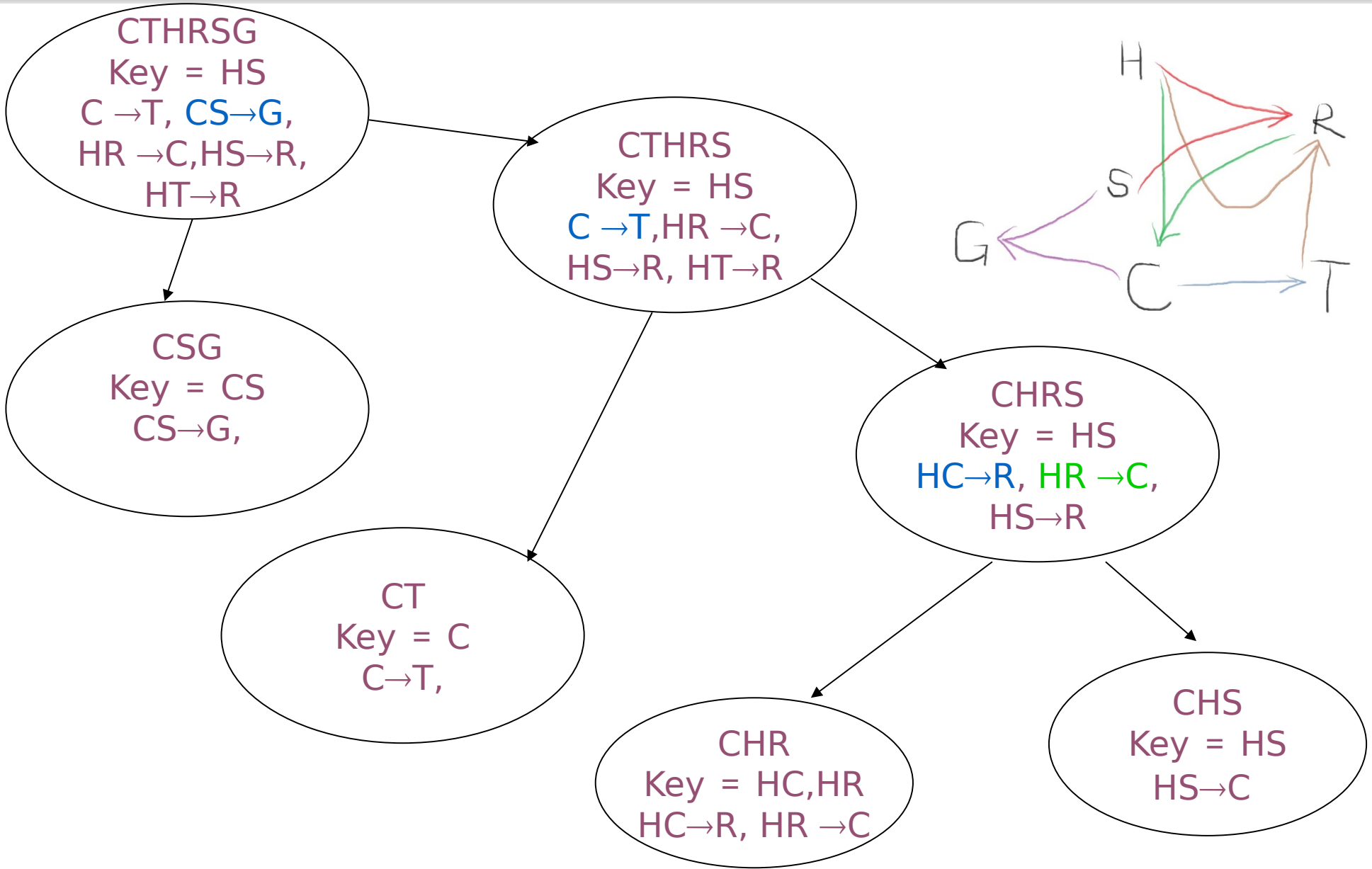
键为

HS

- 分解成 BCNF 模式集的算法的缺点是涉及 F+ 的计算，这一计算的复杂性是指数型的，有人证明判断一个关系模式是否属性 BCNF 是一个 NP 完全问题

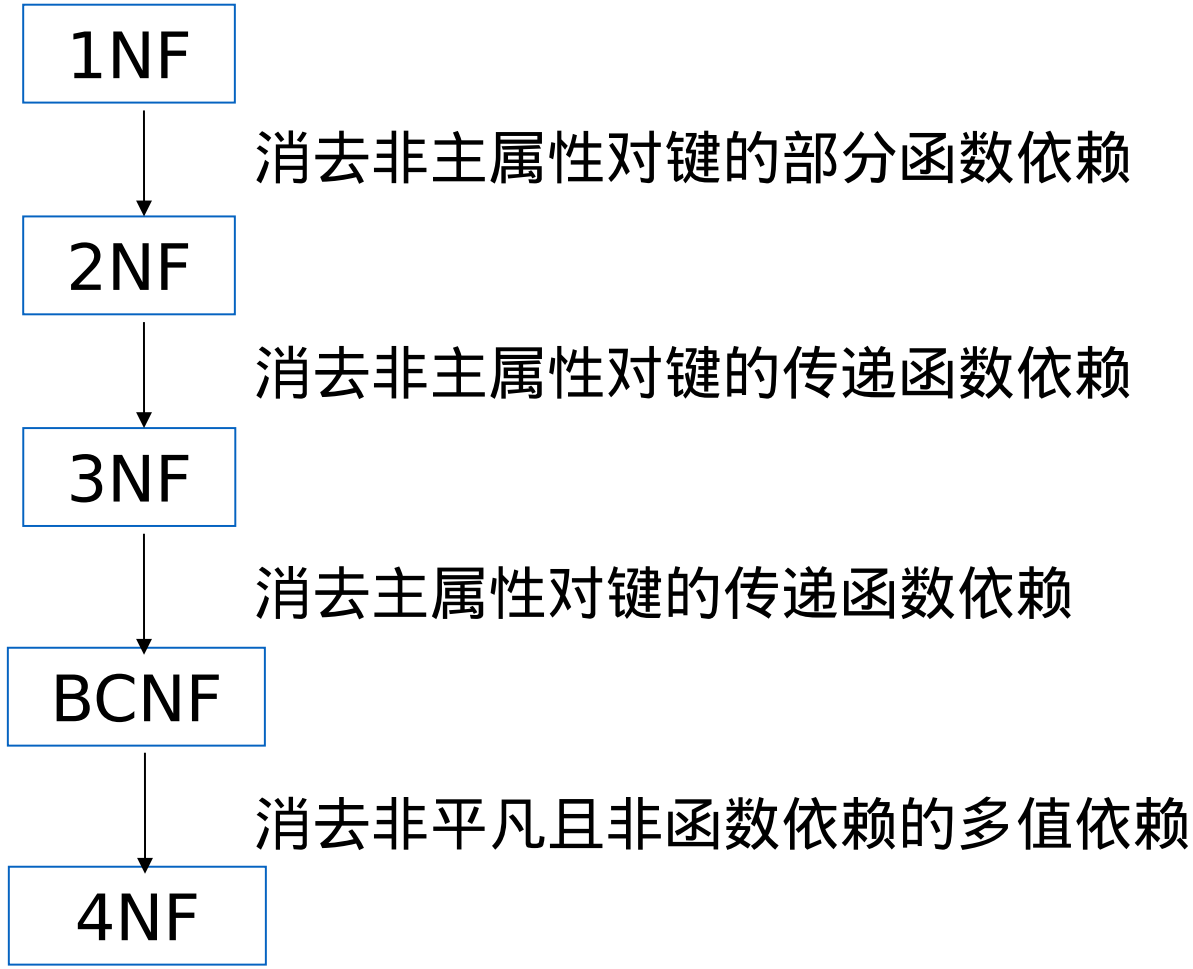
4.4 关系模式的规范化

- 分解树



4.4 关系模式的规范化

- 规范化过程总结



4.4 关系模式的规范化

➢关系模式设计的原则

- 我们在进行关系数据库设计时，应作权衡，尽可能使数据库模式保持最好的特性
- 一般尽可能设计成 BCNF 模式集，如果设计成 BCNF 模式集时达不到保持 FD 的特点，那么只能降低要求，设计成 3NF 模式集，以求达到保持 FD 和无损分解的特点
- 模式分解并不单指把泛关系模式分解成数据库模式，也可以把数据库模式转换成另一个数据库模式，分解和转换的关键是要“等价”地分解
- 一个好的模式设计应符合三条原则：表达性、分离性和最小冗余性